



T.C.
SAKARYA ÜNİVERSİTESİ

BİLGİSAYAR VE BİLİŞİM BİLİMLERİ FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ
PROGRAMLAMA DİLLERİNİN PRENSİPLERİ ÖDEV RAPORU

NÜFUS SİMÜLASYONU

B241210076 - Sinan ULUSINAN

SAKARYA

Nisan, 2025

Programlama Dillerinin Prensipleri Dersi

NÜFUS SİMÜLASYONU

Sinan ULUSINAN^{B241210076*-1A}

Özet

Bu ödevde, Java dili ve Nesne Yönelimli Programlama prensipleri kullanılarak dinamik bir nüfus artışı ve şehir bölünmesi simülasyonu geliştirilmiştir. Bize verilen problem, kullanıcının girdiği başlangıç parametrelerine göre şehir, ilçe, mahalle ve kişi hiyerarşisinin kurulması ve belirli matematiksel kurallara göre alınan bu başlangıç parametreleri ile nüfusun her turda katlanarak artırılmasıdır. Veriler iç içe geçmiş listeler yapısıyla yönetilmiştir. Nüfusu 1000'i aşan şehirler için mitoz bölünme mantığı kurularak ilçelerin yeni şehirlere göç etmesi sağlanmıştır. Gerekli tüm test verileri Java Faker kütüphanesi ile üretilmiştir. Gerekli testler sınıf grubundaki arkadaşlarımın örnek parametreleri ile denenmiş ve herkesçe aynı sonuçlar alınmıştır.

© 2017 Sakarya Üniversitesi.

Bu rapor benim özgün çalışmamdır. Faydalanmış olduğum kaynakları içerisinde belirttim. Herhangi bir kopya işleminde sorumluluk bana aittir.

Anahtar Kelimeler: Nesne Yönelimli Programlama, Simülasyon, Java Faker, Nüfus Modellemesi

1. GELİŞTİRİLEN YAZILIM

Bu ödevde, karmaşık verilerin birbiriyle olan ilişkisini en verimli şekilde yönetebilmek için modüler bir sınıf hiyerarşisi (Şehir => İlçe => Mahalle => Kişi) tasarlanmıştır. Sınıflar, sadece kendi içerdikleri verilerden sorumlu tutulmuş, simülasyonun tüm matematiksel yükü ve yönetimi Oyun sınıfına verilmiştir. Eski C# ödevlerindeki gibi Main sınıfı sadece Oyun sınıfını çağırıp (Manager sınıf) tüm sistemi çalıştırmaktan sorumlu.

1.1. Maven ve Kütüphane Kullanımı: Proje, harici kütüphanelerin olarak indirilmesinin yaratacağı zorlukları aşmak amacıyla sınıf arkadaşlarımın hocalara sorması ve onay alması üzerine Maven mimarisi kullanılarak oluşturulmuştur. Bu sayede pom.xml üzerinden gerekli entegrasyonlar sağlanmış ve projenin taşınabilirliği artırılmıştır.

1.2. Algoritma ve Tasarım Kararları

- Veri Hiyerarşisi:** Kişiler listesi Mahallede, Mahalleler listesi İlçede, İlçeler listesi de Şehirde tutulmuştur. Bu sayede bir mahalleye yeni biri eklendiğinde veya yaşlandırıldığında, üst sınıfların nüfus hesaplamaları otomatik olarak güncel kalmıştır. Yaslan metodu en alttan en üste kadar yaslan() ve yaslanAll() şeklinde gitmiştir. En üst sınıfa, yani Şehir sınıfına “herkesi yaşlandır” emri gelince en alttaki Kisi sınıfına kadar sıra sıra giderek tüm kişileri yaşlandırıyor bu hiyerarşi.
- Nüfus Artışı:** Turlar sırasında nüfus artış oranı, ilgili şehrin mevcut nüfusunun birler ve onlar basamağının toplamı ile dinamik olarak hesaplanmıştır. Her sınıfta ayrı ayrı nüfus verisi tutacaktım başta fakat sonrasında hepsinde getNüfus() gibi bir metod kullanarak daha dinamik bir toplam nüfus bulma komutu ayarladım. Böylelikle ekstra her sınıf için bir “nüfus” verisi tutmak yerine genel dinamik bir nüfus verisi tutulur oldu.

* Ödev Sorumlusu. Sinan ULUSINAN, B241210076

Mail Adresi: sinan.ulusinan@org.sakarya.edu.tr

- **Şehirlerin Mitoz Bölünmesi:** Bir şehrin nüfusu 4 basamaklı sayılara veya dümdüz 1000 ve üzerine ulaştığında Oyun sınıfı devreye girmiş, ilçeleri (ve ilçelerindeki mahalle/nüfusları) yeni bir şehre aktararak dizinin sonuna eklemiştir. Bu süreçte yine hocaların doğrulaması üzerine 1 ilçeli veya 1 mahalleli şehirleri bölerken yine 1 ilçe ve 1 mahalle olarak oluşuyor yeni şehir. Bu sebeple de sanki mitoz bölünmedeki çekirdeğin aynı kalması gibi olduğundan mitoz bölünmeye benzettim bu sistematigi.
- **Önce sözde kod sonra tasarım:** Ödev dosyasını okuduktan sonra her defasında dönüp ne yapmam lazım diye bakmamak için ödevi kendime anlayabileceğim şekilde anlatıp bunun bir notunu aldım. Şekil 1'de gösterilen metin benim ödevi anlamak için uğraştığım 3 saatlik bir süreci temsil ediyor. Daha sonrasında da her sınıfı tek tek ne yapmam lazım diye algoritmasını kurup en küçük sınıf olan Kisi sınıfından başlayarak en üste kadar sıra sıra ekleme yaparak tüm kodları hazırlamış oldum.

```

Oyunda her turun sonunda şehir nüfusları gösterilmelidir , her satıra 5 şehir gelecek şekilde düzenlenmeli. 8 il varsa az önce o yüzden 5 yukarda 3 aşağıda olacak. Ve az önceki şartları gerçekleştirdiğimizde olacak ve ilk tura başlamadan olan nüfus durumu su şekilde olur :
[24]-[30]-[77]-[45]-[66]
[88]-[21]-[55]

daha sonrasında ilk tur başlar. Tur sonrasında yapılacaklar :
* katsayı : nüfus değerlerin basamakları toplanacak (24 + 6)
* bu değer , tüm mahallenin nüfusunun kaç kat olacağını gösteriyor
* mahallelerdeki kisi sayısını bu yeni katsayı sayısıyla çarpacağız (24 sayısını için 1 kişiydi , 24x45 oluşan yeni sayı de bütün toplam nüfusu arttıracak katsayımız.
**endi ise mahalle sayısı ile bu yeni katsayıya çarpıp yeni nüfusu bulacağız (18 sayısından dolayı 8 mahalle , 8x18=144 yeni nüfusu)
*Eğer bir sebepten bu basamakların toplamı sıfır olursa, o şehirdeki her mahallede nüfus sadece 1 kişi artar.
Turun yeni yapı bitti bir şekilde bu mahalle devam edecek ve nüfus şehirlerin nüfusları arasında görünecektir.
* Eğer her turun sonunda emzilenmelidir. Her turden sonra devam etmek için tura bastırarak ilk turun sonuçlarını silirdiribiliriz. Mesela baslangıç değerleri yazdı , tur 1 gerçekleştirdi. Tura basında baslangıç değerleri yerine tur 1 değerleri yukarda yazdı sonra tur 2 değerleri yazdı. böyle böyle istenilen tura kadar devam ederiz. Eğer tur kalmadıysa da oyun sonu diye -----

Bir turun sonuna geldiğinde tüm şehirlerin nüfuslarına bakıyoruz. Eğer herhangi bir ilin nüfusu 1000 veya daha fazlaysa (yani 4 basamaklıysa):
* yeni oluşan şehre Java Fakir kutuphanesi kullanılarak rastgele yeni bir isim veriliyor. Eski şehrin ismi aynı kalıyor.
* İlçeler bölünüyor . Eğer bölünecek şehirin ilçe sayısı çift ise: İlçeler tam ortadan ikiye bölünüyor. Örn: 8 ilçe varsa 4'ü kalıyor, 4'ü yeni şehre gidiyor).
* Eğer bölünecek şehirin ilçe sayısı tek ise: Yeni şehre giden ilçe sayısı 1 eksik oluyor. Yani tek ise 1 arttırıp 2 ye bölüp sonra 1 azaltarak gönderiyorsuz sonraki ilçeye. Daha kolay bir hesaba da olabilir (Örn: 7 ilçe varsa, 4 ilçe eski şehirde kalıyor, 3 ilçe yeni şehre tasınıyor)
* İlçeler kanunlarına bağlı mahalle ve nüfusları da alıyor. Bu sebeple de aslında nüfus 2 ye bölünmüş gibi oluyor. Ama il için bir sayı oluşturmazsakız. Doğrultki sayılar gibi bir sayı olmayacak o yüzden sadece ilçeler bölünecek gibi düşünülmelidir.
* yeni ilçe listenin sonuna ekleniyor.
** mesela 1078 nüfuslu tek ilçeli , oyun başındaki 79 sayısından. 7 ilçe.
1078 / 7 = 154 (ilçe başına nüfus). 7 olduğundan 4 ve 3 ilçe olarak ayrılacak. 4x154 ve 3x154 ilemlinden eski şehir=636 kalır , yeni şehir=462 olarak oluşturulur.
** mesela 1400 nüfuslu çift ilçeli. oyun başında 80 sayısından. 8 ilçe.
1400 / 8 = 175 (ilçe başına nüfus). 8 olduğundan 4 4 bölünür. 4x175=700. eski ve yeni şehir aynı nüfusla olur.

-----

sistem hiyerarşisi:
oyun sınıfı : tüm simülasyona yönetecek
şehir sınıfı: ad , toplam nüfus ve ilçelerin listesi
ilçe sınıfı : ad , nüfus ve ilçeye bağlı mahalle listesi
mahalle sınıfı: ad , nüfus ve mahallede yaşayan kişilerin listesi
kisi sınıfı: adı/soyadı , yaş , ID (int türde)
* fakir ile oyun başı her kişiye 0-10 arası yaş atacak. her tur bitiminde simülasyondaki herkesin yaşı 1 artacak.
* nüfus artınca forçüble içinde fakirleri tekere çekilip bu insanlara yeni ilde ve soy isimler verileceği. Yeni gelen nüfusluları de yeni doğmuş gibi düşünüp 0 yaşında gibi kabul edip tur sonunda her türlü 1 yaşına basmış olurlar.
* ID değeri eşsiz olacak , kisi sınıfında static sayar yapıp her yeni kişide ID 1 arttırarak ilerlemek en mantıklısı yeni gelen insanlar için de bu sayar çok kolay hayatı kurtarır.

- oyun bitiminde en son ekranda son şehrin nüfus durumları yazacak.
- satır sütun seçimi : şehirleri matris gibi düşün , 8 dan başlayarak satır sütunlardan bir değer isteyebilirsin. satır ve sütun değerine göre istenilen şehir seçilmiş olacak
- seçim simülasyonu tüm veriler ekrana düşülecek. format şöyle:
ŞEHİR : ad - nüfus : değer
İLÇE : ad - nüfus : değer
MAHALLE : ad - nüfus : değer
KİŞİLER :
ID-ad-yaş
ID-ad-yaş
... diğer kişiler
MAHALLE : ad - nüfus : değer
KİŞİLER:
ID-ad-yaş
ID-ad-yaş
... diğer kişiler
... diğer mahalleler

```

Şekil 1. TXT formatındaki notlarımı

2. ÇIKTILAR

Yazılım öncelikle tur sayıları 3'ten fazla ve girilen sayılar da 5'ten fazla olursa biraz yavaşlıyor. sistemdeki nesne sayısı katlanarak arttığı için RAM tüketimi artmakta ve performans düşüşü yaşanmaktadır. Fakat temelde yazılımda 2 tane çıktı var.

2.1 Matematiksel Çıktılar: Kullanıcıdan alınan parametreler sonucu şehirlerde ne kadar nüfus olduğuna dair bir çıktı alıyoruz her tur sonunda. Kullanıcının istediği tur sayısı bittikten sonra da oyun sonu ekranı olarak en sondaki tur sonucu ekrana geliyor. Buraya kadarki yapılan tüm işlemler matematiksel ve aritmetik işlemlerden geçiyor ve bize 5'li sıralanmış bir matris halinde ekrana yansıtılıyor.

2.2 Yaşayan İnsanların Çıktısı: Simülasyon bittiğinde, yani en sondaki 5'li sıralanmış matrisler geldikten sonra kullanıcının girdiği satır-sütun değerine karşılık gelen şehrin detaylı hiyerarşik dökümü (Şehir => İlçe => Mahalle => Kişiler [ID-İsim-Yaş]) ekrana hatasız yazdırılmıştır. Burada eğer birden fazla ilçe ve mahalle varsa yine dinamik olarak alt alta gösterilmiştir.

3. SONUÇ

Bu çalışma sonucunda; Java'da ArrayList yapılarının dinamik yönetimi, sınıflar arası veri aktarımı ve Maven ile proje yapılandırması konularında derinlemesine pratik yapılmıştır. Gerçek hayatta da çok katmanlı kurumsal yazılımların veya oyun motorlarının alt yapısında kullanılan "Tek Sorumluluk Prensiplerinin" koda sağladığı temizlik ve yönetilebilirlik avantajı birebir tecrübe edilmiştir. Şahsi olarak en büyük katkısı ise geçtiğimiz dönemlerde tam olarak anlamadığım OOP konusunu kod üzerinde ve sınıflar üzerinde daha net ve pratik ile anlamam olmuştur. Diğer en büyük katkısı da kendim üzerinde uğraşmak istediğim bilgisayar oyunlarında yapılacak olan düzen ve sistematik için güzel ve zorlu bir ödev oldu.

Özellikle şehir bölünmesi algoritmasını tasarlariken yaşanan zorluklar, nesnelerin referans tipli olmasının önemini kavramamı sağlamıştır. Sınıfları başta ne şekilde görevlendirmeliyim diye cebelleşirken en küçük sınıftan başlayıp kendi notlarıma bakıp gerekenleri sıra sıra yazdığım sırada yavaş yavaş 2 3 sınıfı bitirdiğimi görmüş oldum. Daha sonrasında test aşamasında sınıf grubunda paylaşılan “benim bu parametreler ile sonucum bu oldu sizin de aynı mı?” diye paylaşılan örnek veriler ile denediğim parametrelerden aynı sonuçları alamadığımda ufak korkular yaşadım. Fakat sınıf grubundaki insanların yardımı ile ufak tefek mantık hatalarım olduğunu fark edip onları düzelttim ve kodu herkesinki gibi çalışır hale getirdim. Eclipse programını , Java dilini ve OOP'yi anlamam için zevkli ve güzel bir ödev oldu.

Referanslar

[1] Sakarya Üniversitesi, Programlama Dillerinin Prensipleri Dersi Ödev Yönergesi, 2024.

[2] Java Faker Documentation, <https://dius.github.io/java-faker/>